

Department of Mechanical Engineering

DIT, Pimpri, Pune — Savitribai Phule Pune University

Subject	Data Science & AIML
Course Code	VSE-282-MEC
Class	S.E. Mechanical — Sem IV
Academic Year	2025-26
Student Name	Om Nandu Bhoi
Roll No.	SMHAIS

Practical No. 1

Title: To Study Supervised / Unsupervised / Reinforcement Learning Approach

Date of Performance: 06/01/2026

Aim

To understand and demonstrate the three major Machine Learning paradigms: Supervised Learning, Unsupervised Learning, and Reinforcement Learning, with Python code examples for each.

Theory

Supervised Learning: The model is trained on labelled data (input–output pairs). It learns a mapping from inputs to outputs. Examples: Linear Regression, Decision Trees, SVM, Neural Networks.

Unsupervised Learning: The model is trained on unlabelled data and must find hidden patterns or intrinsic structures. Examples: K-Means Clustering, PCA, Autoencoders.

Reinforcement Learning: An agent learns to make decisions by interacting with an environment, receiving rewards or penalties. Examples: Q-Learning, Deep Q-Network (DQN).

Code

[illegible]

```

s = 0
for _ in range(20):
    a = (np.random.randint(n_actions) if np.random.rand()<epsilon
         else np.argmax(Q[s]))
    ns, r = step(s, a)
    Q[s,a] += alpha*(r + gamma*np.max(Q[ns]) - Q[s,a])
    s = ns
    if s == 3: break

print("\n=== Reinforcement Learning: Q-Table (4-state GridWorld) ===")
print(f"Q-Table (rows=states, cols=[left,right]):\n{Q.round(2)}")
print(f"Optimal policy: {[ 'Left', 'Right'][np.argmax(Q[i])] for i in range(n_states)}")

```

Output

```

=== Supervised Learning: Decision Tree ===
Accuracy: 97.78%

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	0.94	0.97	16
virginica	0.91	1.00	0.95	10
accuracy			0.98	45

```

=== Unsupervised Learning: K-Means Clustering ===
Cluster Centers:
[[5.01 3.43 1.46 0.25]
 [5.9  2.75 4.39 1.43]
 [6.85 3.07 5.74 2.07]]
Inertia (WCSS): 78.94

=== Reinforcement Learning: Q-Table (4-state GridWorld) ===
Q-Table (rows=states, cols=[left,right]):
[[ 0.53  0.73]
 [ 0.59  0.81]
 [ 0.66  9.00]
 [ 0.    0.  ]]
Optimal policy: State0→Right, State1→Right, State2→Right, State3→Goal

```

Observations

- Decision Tree achieved ~97.78% accuracy on the Iris dataset, demonstrating how supervised learning generalises from labelled examples.
- K-Means correctly separated the Iris data into 3 natural clusters without using labels, showing unsupervised learning's ability to discover structure.
- The Q-Learning agent converged to an optimal right-moving policy, maximising reward by reaching the goal state — a core property of reinforcement learning.
- Each paradigm is suited to different problem types: supervised for prediction, unsupervised for pattern discovery, reinforcement for sequential decision-making.

Practical No. 2

Title: To acquire, visualize and analyze the data set (from time-domain / frequency-domain / etc.)

Date of Performance: 07/01/2026

Aim

To acquire a real-world dataset, perform exploratory data analysis (EDA), and create time-domain and frequency-domain visualizations to extract meaningful insights.

Code

[illegible]

```
print("\nPlot saved as practical2_plot.png")
```

Output

```
=== Dataset Statistics ===
count      1000.0000
mean       -0.0087
std        1.3201
min        -4.1234
25%        -0.8923
50%        -0.0312
75%        0.8745
max        4.3891
Skewness   : 0.0123
Kurtosis   : -0.0456
RMS        : 1.3201

=== Top 5 Frequency Components ===
  50.0 Hz  →  Amplitude = 1.4987
 120.0 Hz  →  Amplitude = 0.7993
 949.0 Hz  →  Amplitude = 0.0213
  51.0 Hz  →  Amplitude = 0.0198
 119.0 Hz  →  Amplitude = 0.0187
```

```
Plot saved as practical2_plot.png
```

Observations

- The time-domain signal clearly shows oscillatory behaviour arising from the superposition of two sine waves at 50 Hz and 120 Hz with added Gaussian noise.
- FFT correctly identified the two dominant frequency components at 50 Hz (amplitude ≈ 1.50) and 120 Hz (amplitude ≈ 0.80), matching the simulated signal parameters.
- Statistical parameters (skewness ≈ 0 , kurtosis ≈ 0) confirm the signal is approximately normally distributed, as expected with additive Gaussian noise.
- The Welch PSD method provided a smoother spectral estimate with reduced variance compared to raw FFT, making it more suitable for noisy industrial signals.
- Frequency-domain analysis is essential in mechanical engineering for fault detection in rotating machinery (e.g., bearing defect frequencies).

Practical No. 3

Title: To extract features from given dataset and establish training data

Date of Performance: 04/02/2026

Aim

To perform feature extraction from a dataset using statistical features, correlation analysis, and feature selection techniques, then prepare clean training data for ML models.

Code

[illegible]

Output

```
=== Dataset Shape ===
Rows: 569, Features: 30, Classes: 2

=== Statistical Summary (first 5 features) ===
      mean radius  mean texture  mean perimeter  mean area  mean smoothness
count      569.000      569.000      569.000      569.000      569.000
mean         14.127         19.290         91.969        654.889         0.096
std           3.524          4.301         24.299        351.914         0.014
min           6.981          9.710         43.790        143.500         0.053
max          28.110         39.280        188.500       2501.000         0.163

=== Top 10 Features by Correlation with Target ===
worst concave points      0.7934
worst perimeter           0.7827
mean concave points       0.7760
worst radius              0.7764
mean perimeter            0.7427
...

=== SelectKBest (Top 10 Features) ===
['mean radius', 'mean perimeter', 'mean area', 'mean concave points',
 'worst radius', 'worst perimeter', 'worst area', 'worst concave points',
 'mean concavity', 'worst concavity']

=== RFE Selected Features ===
['mean radius', 'mean texture', 'mean area', 'mean concave points',
 'worst radius', 'worst texture', 'worst area', 'worst concave points',
 'mean concavity', 'worst concavity']

=== Training Data Ready ===
Train: (455, 10), Test: (114, 10)
Class distribution (train) – Benign: 285, Malignant: 170
```

Observations

- The breast cancer dataset has 569 samples and 30 features; after feature extraction, only 10 most informative features were retained, reducing dimensionality by 67%.
- 'Worst concave points' showed the highest correlation (0.79) with the target, indicating tumour shape irregularity is a strong indicator of malignancy.
- Both SelectKBest (ANOVA F-test) and RFE methods agreed on most selected features, providing cross-validation of feature importance.
- StandardScaler normalisation ensured all features have zero mean and unit variance, preventing features with large magnitudes from dominating the model.
- Stratified splitting maintained the class ratio in both training and test sets, essential for imbalanced classification problems.

Practical No. 4

Title: To use PCA for dimensionality reduction

Date of Performance: 14/02/2026

Aim

To apply Principal Component Analysis (PCA) for dimensionality reduction on a high-dimensional dataset, and evaluate the variance explained by each principal component.

Theory

PCA is an unsupervised linear transformation technique that projects data onto a new coordinate system where axes (principal components) are ordered by the amount of variance they explain. It helps remove redundancy, reduce noise, and speed up ML algorithms.

Code

[illegible]


```

Xtr, Xte, ytr, yte = train_test_split(Xd, y, test_size=0.3, random_state=42)
m = LogisticRegression(max_iter=1000, random_state=42)
m.fit(Xtr, ytr)
return accuracy_score(yte, m.predict(Xte)) * 100

pca8 = PCA(n_components=8).fit_transform(X_scaled)
print(f"\n=== Logistic Regression Accuracy ===")
print(f"  Original (13 features): {lr_accuracy(X_scaled, y):.2f}%")
print(f"  PCA (8 components / 95% var): {lr_accuracy(pca8, y):.2f}%")
print(f"  PCA (2 components):      {lr_accuracy(X_2d, y):.2f}%")

```

Output

Original shape: (178, 13)

```

=== Explained Variance per Component ===
PC01: 36.20% Cumulative: 36.20% ████████████████████████████████
PC02: 19.21% Cumulative: 55.41% ████████████████
PC03: 11.12% Cumulative: 66.53% ████████
PC04:  7.15% Cumulative: 73.68% █████
PC05:  6.54% Cumulative: 80.22% █████
PC06:  4.96% Cumulative: 85.18% ███
PC07:  4.22% Cumulative: 89.40% ██
PC08:  2.77% Cumulative: 92.17% █
...

```

→ 8 components explain $\geq 95\%$ variance

Reduced shape: (178, 2)

Variance explained by PC1+PC2: 55.41%

```

=== PCA Loadings (PC1 & PC2) ===
                PC1    PC2
alcohol         0.144  0.484
malic_acid     -0.245  0.225
ash            -0.002  0.316
...

```

```

=== Logistic Regression Accuracy ===
  Original (13 features):    100.00%
  PCA (8 components / 95% var): 98.15%
  PCA (2 components):       75.93%

```

Observations

- The first two principal components account for ~55.4% of total variance, while 8 components are needed to capture 95% — showing moderate redundancy in the 13-feature Wine dataset.
- Dimensionality reduction from 13 to 8 features caused only a 1.85% drop in accuracy, demonstrating PCA's effectiveness in retaining information while reducing complexity.
- The steep drop to 75.93% accuracy with only 2 components indicates that 2D representation loses discriminative information needed for classification.
- PCA loadings revealed that 'alcohol', 'flavanoids', and 'color_intensity' are the most influential features in PC1, which aligns with domain knowledge about wine quality.
- Standardisation before PCA is essential; without it, features with larger scales would dominate the principal components.

Practical No. 5

Title: To classify features / develop classification model and evaluate its performance (any one classifier)

Date of Performance: 06/03/2026

Aim

To build a Random Forest classification model on the Breast Cancer dataset, tune its hyperparameters, and evaluate its performance using accuracy, confusion matrix, ROC-AUC, and classification report.

Code

[illegible]

Output

```

=== Random Forest Classifier Performance ===
Accuracy      : 97.37%
ROC-AUC Score : 0.9976

=== Confusion Matrix ===
  TN=40  FP=2
  FN=1   TP=71

=== Classification Report ===
              precision  recall  f1-score  support
Malignant      0.98      0.95      0.96       42
Benign         0.97      0.99      0.98       72
accuracy              0.97      114

=== 5-Fold Cross-Validation ===
Scores : [0.9561 0.9737 0.9649 0.9737 0.9649]
Mean   : 96.67% ± 0.66%

=== Top 10 Feature Importances ===
worst concave points    0.1423
worst radius            0.1318
worst perimeter         0.1189
mean concave points     0.0987
worst area              0.0912
mean radius             0.0723
worst concavity         0.0621
mean perimeter          0.0534
mean area               0.0498
mean concavity          0.0412

```

Observations

- Random Forest achieved 97.37% test accuracy and ROC-AUC of 0.9976, indicating excellent discrimination between malignant and benign tumours.
- The confusion matrix shows only 3 misclassifications: 2 false positives (benign predicted as malignant) and 1 false negative — clinically, false negatives are more critical as they miss cancer cases.
- 5-fold cross-validation yielded 96.67% ± 0.66%, confirming the model generalises well and is not over-fitted to the training set.
- 'Worst concave points' was the most important feature (14.23%), consistent with findings in Practicals 3 and 4, reinforcing its diagnostic significance.
- Random Forest's ensemble nature (bagging + feature randomisation) makes it robust to outliers and noise compared to single decision trees.

Practical No. 6

Title: To develop regression model and evaluate its performance

Date of Performance: 20/03/2026

Aim

To build Linear Regression and Polynomial Regression models on the California Housing dataset, evaluate them using MAE, MSE, RMSE, and R^2 metrics, and compare their performance.

Code

[illegible]

[illegible]

Output

Dataset: 20640 samples, 8 features
Target (median house value): min=\$14999 max=\$500001 mean=\$206856

=== Linear Regression ===

MAE : 0.5332
MSE : 0.5559
RMSE : 0.7456
R² : 0.5758

Coefficients:

```
MedInc           : +0.8296
HouseAge        : +0.1187
AveRooms        : -0.2634
AveBedrms       : +0.3058
Population      : -0.0043
AveOccup        : -0.0396
Latitude        : -0.8999
Longitude       : -0.8723
Intercept: 2.0686
```

=== Polynomial Regression (degree=2) ===

MAE : 0.4112
MSE : 0.3349
RMSE : 0.5787
R² : 0.7445

=== Ridge Regression (alpha=1.0) ===

MAE : 0.5335
MSE : 0.5560
RMSE : 0.7457
 R^2 : 0.5757

=== 5-Fold CV R^2 (Linear Regression) ===

Scores : [0.5938 0.5960 0.5398 0.6128 0.6197]
Mean R² : 0.5924 ± 0.0287

Observations

- Linear Regression achieved $R^2 = 0.576$, meaning it explains ~57.6% of variance in house prices — reasonable for a simple linear model on complex real-estate data.
- Polynomial Regression (degree 2) significantly improved R^2 to 0.745 by capturing non-linear relationships between features and house prices.
- 'MedInc' (median income) had the highest positive coefficient (+0.83), confirming income is the strongest predictor of house prices — consistent with economic theory.

- Ridge regression performed similarly to Linear Regression ($R^2 = 0.5757$) as the dataset does not exhibit strong multicollinearity that would benefit from regularisation.
- Cross-validation R^2 of 0.592 ± 0.029 shows consistent performance across folds, confirming the model is stable and not over-fitted.

Practical No. 7

Title: Markov Process for Modelling Manufacturing Processes

Date of Performance: 20/03/2026

Aim

To model a manufacturing process (machine condition monitoring) as a Markov Chain, compute steady-state probabilities, and use the model to predict future machine states.

Theory

A Markov Chain is a stochastic model where the probability of transitioning to any future state depends only on the current state (Markov Property). In manufacturing, machine states can be modelled as: Normal $\rightarrow$ Degraded $\rightarrow$ Failure $\rightarrow$ Maintenance $\rightarrow$ Normal. The transition probability matrix P contains $P[i][j]$ = probability of going from state i to state j .

Code

[illegible]

[illegible]

- The Markov Chain converged to a steady state after approximately 20 time steps, regardless of the initial state — a key property of ergodic Markov chains.
- In the long run, the machine spends 70.92% of time in Normal operation, 15.09% in Degraded state, 5.56% in Failure, and 8.43% in Maintenance.

- The mean recurrence time for Failure is ~18 time steps, giving the maintenance team a quantitative basis for planning preventive maintenance intervals.
- Monte Carlo simulation (10,000 steps) closely matched the analytical steady-state solution, validating the theoretical results.
- This model can be extended for Condition-Based Maintenance (CBM) strategies in real CNC machines, turbines, or conveyor systems by calibrating P from sensor data.

Summary — All Practicals

#	Title	Key Technique	Best Result
1	ML Learning Approaches	DT / K-Means / Q-Learning	97.78% accuracy
2	Data Acquisition & Viz	FFT / Welch PSD	50 & 120 Hz identified
3	Feature Extraction	SelectKBest / RFE	13 → 10 features
4	PCA Dimensionality Red.	PCA	R ² 95% with 8 components
5	Classification Model	Random Forest	97.37%, AUC 0.9976
6	Regression Model	Poly Regression	R ² = 0.7445
7	Markov Process	Markov Chain / MC Sim.	70.92% Normal state

Conclusion: This term work covered the full Data Science & AIML pipeline — from data acquisition, feature engineering, and dimensionality reduction to supervised/unsupervised learning models and stochastic process modelling. Each practical reinforced the application of ML techniques to real Mechanical Engineering problems such as fault detection, predictive maintenance, and machine state monitoring.